

INTERNAL PREP DOCUMENT

Judge Q&A — Technical Deep-Dive

Anticipated technical and product questions, organized by theme, with straight answers — including the honest limitations. A judge who probes will respect a specific, confident answer over a polished dodge. Use this to rehearse, not to read from on stage.

34

questions covered

8

theme sections

24

real backend tests, for reference

A Architecture & Stack

5 questions

Q Walk us through your tech stack, end to end.

React 19 + TypeScript + Vite on the frontend, Tailwind v4 for styling, Framer Motion for motion. The backend is FastAPI with SQLAlchemy's async ORM over Postgres, migrations via Alembic. Postgres and file storage both run on Supabase; the backend deploys on Render, the frontend on Vercel. AI runs through Groq's OpenAI-compatible API (**gpt-oss-120b**) and YarnGPT for text-to-speech — both called directly over HTTP, no vendor SDKs.

Q Why FastAPI over Node/Express or Django?

Async-native by default, which matters because almost every request in this app fans out to an external call — Postgres, Groq, YarnGPT, Supabase Storage. FastAPI plus Pydantic gave us type-safe request/response contracts for free, which mattered more with a five-person team moving fast than raw throughput would.

Q Why Supabase instead of running your own Postgres and object storage?

A managed Postgres with a connection pooler we don't have to operate, plus object storage in the same project, on a free tier realistic for an MVP. The one real gotcha: Supabase's direct database host is IPv6-only and unreachable from Render, so production routes through Supavisor's pooler instead.

Q Is this a monorepo? How do frontend and backend relate?

One repo, two independently deployable halves — **/src** for the Vite app, **/backend** for FastAPI. They only talk over HTTP. In dev, Vite proxies **/api** to localhost so there's no CORS friction locally; in production, CORS is locked to the exact Vercel origin, nothing wildcarded.

Q Walk me through what actually happens when a parent logs in.

They enter a school code and phone number. The frontend calls a real endpoint, **GET /schools/lookup/{code}**, which looks the school up in Postgres and returns its public profile. There's no password on this side — that's an intentional trust-model decision, not an oversight, and it's covered in the limitations section.

B AI & LLM Integration

6 questions

Q Which model are you using, and why?

Groq's **gpt-oss-120b**, an open-weight reasoning model, via Groq's OpenAI-compatible chat completions endpoint. Two reasons: Groq's inference is fast enough to feel live inside a UI, and in our own testing, gpt-oss-120b handled Yoruba, Igbo, and Hausa noticeably better than smaller models we tried first.

Q Is the multilingual output real generation, or translation of a canned English response?

Real generation, directly in the target language, from the underlying data. There's no English draft and no translation pass — the system prompt tells the model the target language and it writes the summary in that language from scratch, every time.

Q How do you stop the AI from hallucinating facts about a child?

The system prompt explicitly instructs it to use only the data it's given, and to say "I don't have that information" rather than guess. Structurally, every call passes the real pupil/workload data as JSON — there's nothing else for it to draw from, so there's nothing to hallucinate about.

Q Is the AI response cached, or generated fresh every time?

Fresh, on demand — there's a "Get AI insight" button rather than auto-generation, specifically to control cost and latency on something billed per call. Caching identical requests is the obvious next optimization; we scoped it out of the fellowship timeline on purpose.

Q What happens if Groq is slow or down?

Every call is wrapped in explicit error handling — a timeout or bad response returns a clean error message, never a stack trace or a hang. We hit a real version of this during build: as a reasoning model, gpt-oss-120b sometimes burned its entire token budget on internal reasoning and returned *empty* content, silently, with a 200 status. Fixed with a **reasoning_effort** parameter and a higher token ceiling — a real bug we found and fixed, not a hypothetical.

Q Why not just call the OpenAI API?

Cost and language quality, mainly — Groq's pricing and speed made an on-demand, per-click AI feature viable, and gpt-oss-120b's Nigerian-language output tested better for us than GPT-4o-mini on the same prompts. It's also provider-agnostic at the code level: we call raw HTTP, not a vendor SDK, so switching providers is a config change, not a rewrite.

C Voice & Multilingual (TTS)

4 questions

Q How does the voice summary actually work?

A real text-to-speech API, YarnGPT, called server-side with the AI-generated summary text. It returns real MP3 audio, which streams to an **<audio>** element in the browser — it's not the browser's built-in speech synthesis, it's a dedicated Nigerian-voice TTS model.

Q Does it actually support Yoruba, Igbo, and Hausa audio, or just English?

Tested live, end to end, against English and Yoruba text — real audio files generated and verified. YarnGPT's voices aren't language-locked; they're trained across Nigerian languages, which is the whole reason we chose it over a generic TTS provider.

Q What's the latency, and how do you hide it in the UI?

Roughly 15 to 30 seconds for a full paragraph — real API latency we can't shrink from our side. The button shows a "Loading audio..." state instead of blocking silently, and on stage, we narrate through the wait instead of standing in dead air.

Q What happens if YarnGPT fails or isn't configured?

It fails gracefully in both cases — a clear "not available yet" message before it was wired up, and a clean retry message if the live call errors. The frontend never shows a raw error or a broken audio player.

D Auth, Security & Data Trust

5 questions

Q How is school data protected?

School admin accounts are real: bcrypt-hashed passwords, JWT session tokens (HS256, 7-day expiry) issued on login and required on every write. Every query is scoped to the authenticated school's ID — one admin can never read or post into another school's data, and that's covered by the automated test suite, not just assumed.

Q The parent side has no login — isn't that a security hole?

It's a deliberate trust model, not an oversight: knowing a school's code is the same as being handed it by the school, no different from a class WhatsApp invite link. Nothing sensitive — no grades, no payment data — sits behind it today, and it's the first thing we'd harden before a real launch. The API layer is already separate from the UI, so adding parent auth later doesn't require a rewrite.

Q Where do your API keys live? Can the browser ever see them?

Server-side only, in environment variables. Groq and YarnGPT calls happen entirely inside FastAPI — the frontend only ever talks to our own API, never directly to either third-party provider.

Q Is there rate limiting on the AI endpoints?

Not yet — an honest gap. Nothing currently stops repeated calls from running up the Groq/YarnGPT bill. It's a small addition, a per-IP or per-session request cap, that we scoped out of the fellowship timeline but would add before any real traffic.

Q Is pupil data encrypted?

Today, pupil and workload data is mocked fixture data, not real student records, so this doesn't yet apply. The layer that *is* real — school accounts and posted updates — sits in Postgres over TLS via Supabase's pooler; nothing is ever stored in plaintext files or client-side.

E Testing & Reliability

3 questions

Q What's your test coverage like?

24 backend tests, all passing, covering auth, school registration, update posting and scoping, file storage, and every AI endpoint — insight generation, chat, and text-to-speech. Every external call — Postgres, Supabase Storage, Groq, YarnGPT — is mocked in the suite, so tests never hit real paid APIs or touch real data.

Q How do you know the mocks aren't hiding a broken real integration?

We didn't trust the mocks alone. Every AI feature was verified by hand against the live Groq and YarnGPT APIs before we called it done — real Yoruba and Hausa text, real generated audio files checked with a file-type tool, not just an HTTP 200. We also ran full browser automation against the actual running app, which is how we caught a broken import that unit tests alone had missed.

Q What's your CI/CD setup?

Render and Vercel both auto-deploy on push to **main**. There's no CI gate yet that blocks a bad deploy on a failing test — today that's a manual discipline, not an automated one. That's the next infrastructure investment, not a technical blocker.

F Deployment & Infrastructure

3 questions

Q Is this actually live right now, or running on someone's laptop?

Live — backend on Render, frontend on Vercel, database on Supabase. What you're watching hits the internet, not localhost, over whatever wifi is in the room.

Q What's your uptime story on free-tier hosting?

Honestly — Render's free tier cold-starts after 15 minutes idle, so the first request after a gap can take a few seconds to wake the server. That's a limitation of the tier, not the architecture; a paid tier removes it with zero code changes.

Q How would this scale to 10,000 schools?

The backend is stateless — sessions live in JWTs, not server memory, so instances replicate horizontally behind a load balancer with no code change. Postgres would move off the free-tier pooler to a dedicated instance with read replicas, and AI calls would need caching and a queue instead of synchronous request/response. All additive, not a rearchitecture.

G Scalability & Cost

4 questions

Q What does this actually cost to run, per school or per parent?

Fixed infra cost is effectively zero today — everything's on free tiers. The variable cost is AI: a Groq insight call is a fraction of a cent; YarnGPT's voice synthesis is the more expensive of the two per call. At real scale, the first lever is caching identical insight requests instead of regenerating on every click.

Q Why not build this as SMS/USSD, given many target users may not have smartphones or data?

That's the direction the voice feature already points toward — audio-first access is our accessibility answer today, and the AI and data layers already sit behind a clean API, separate from the UI, so an SMS/USSD channel could be added without touching either. We built the smartphone app first because it validated the AI and multilingual value proposition fastest within the fellowship timeline.

Q What's your realistic path to more languages beyond Yoruba, Igbo, and Hausa?

A config change, not new integration work. Groq's model already understands more languages than we expose in the UI, and YarnGPT's voice roster isn't hard-coded to four languages. Adding one is adding an option to a dropdown and an entry to a lookup table.

Q What's your business model, briefly, from a technical-cost lens?

Schools are the paying customer; parents join free via a school code, which is also our distribution loop — every school that signs up brings its parent base with it. The AI cost per school scales with usage, not headcount, so pricing per school naturally tracks the cost driver.

H Tough / Honest-Limitation Questions

5 questions

Q What's fake in this demo? Be honest.

Pupil, workload, and attendance data is mocked fixture data — not connected to a real school's student information system yet. Everything else — school accounts, authentication, posted updates, and every AI response you see or hear — is real and live. We deliberately faked the layer that would need a real school data-sharing agreement to be genuine, and kept everything a judge can verify live — the AI, the voice, the backend — real.

Q If I sign up as a new school right now, does it actually work?

Yes — try it. Real registration, a real bcrypt-hashed password, a real unique school code generated server-side, immediately queryable by any parent. Nothing about that path is staged for the demo.

Q What's the single biggest technical risk in this system right now?

Dependence on two external paid APIs we don't control. If Groq or YarnGPT change pricing, rate-limit us, or go down, the AI features degrade. We mitigated the *failure mode* — clean errors, no crashes — but not the dependency itself. Multi-provider fallback is the honest next step.

Q How long did the AI integration take, and what actually broke?

About a day of focused work. The real bug: the reasoning model spends part of its token budget "thinking" before writing an answer, and with a low ceiling it sometimes burned the whole budget reasoning and returned nothing — silently, with a 200 status, no error thrown. We only caught it by testing non-English prompts specifically, then fixed it with a model parameter and a higher token budget. It's why we manually verify every AI path against the real API instead of trusting a green test suite alone.

Q Could a competitor copy this in a weekend?

The individual pieces — an LLM call, a TTS call — sure, anyone can wire those up quickly. Harder to copy: the specific combination aimed at Nigeria's actual literacy and language reality, tuned through real testing against real Nigerian-language output, plus the school-network distribution model that gets it in front of users at all. The moat is the product decision, not the API call.

If you get a question you genuinely don't know

Never bluff a technical detail — judges who ask sharp questions can tell. Bridge instead: "I don't have that number memorized, but here's how we'd think about it—" and reason from what you do know. A confident, honest "we haven't solved that yet, and here's our plan" beats a shaky guess every time.